Defining a Class

CS300: Programming II

Cole Nelson

Midterm #1

This Wednesday at 5:45 pm -- check your email for your exam location.

Covers Modules 01-03. Nothing from this week is on it.

Learning Objectives

1. Be able to **write** a class with private instance fields.
2. Be able to **write** a constructor that initializes every field.
3. Be able to **use** `this` to disambiguate a field from a parameter.
4. Be able to **differentiate** overloaded constructors by parameter list.

Book

Today, we'll write our own `Book`!

Before an object, we used parallel arrays

```
String[] titles  = {"Dune", "Emma"};
String[] authors = {"Frank Herbert", "Jane Austen"};
boolean[] out    = {false, true};
```

Nothing but your own discipline keeps index 1 meaning *Emma* in all three. **Not ideal.**

You have only ever seen the outside

```
Book dune = new Book("Dune");
dune.checkOut();
System.out.println(dune.getTitle());
```

Two weeks of this, and just a glimpse of `Book.java`. That was the point -- a class you use is a set of promises, not a file you read. That is **abstraction**.

Today the file is yours.

Eclipse Exercise

Download today's starter from Canvas and code along.

Run `Main` first. It doesn't compile -- read the error before you fix anything.

One class, one file, one name

```java
// Book.java
public class Book {

}
```

The file name and the class name must match, letter for letter.

Fields are what the object remembers

```
public class Book {
  private String title;
  private String author;
  private boolean checkedOut;
  private int daysLeft;
}
```

Declared, not assigned. Every `new Book(...)` gets its own fresh set of these four.

Why `private`?

Because you get to decide what a `Book` will *let* someone do to it. A `public` field is a promise that anyone, anywhere, can set it to anything.

Instead, we often prefer getters and setters; a topic for next time!

Java writes a constructor if you don't

```
public class Book {
  private String title;     // null
  private int daysLeft;     // 0
}
```

```
Book dune = new Book();     // this compiles!
```

You never wrote `Book()`. Java supplies a no-argument one that leaves every field at its default.

...until you write one yourself

```
public class Book {
  public Book(String title, String author) { ... }
}
```

```
Book dune = new Book();    // now this does NOT compile
```

The free one disappears the moment you define any constructor. Java assumes you had a reason.

A constructor has no return type

```
public Book(String title, String author) {
  this.title = title;
  this.author = author;
  this.checkedOut = false;
  this.daysLeft = 0;
}
```

Not `void`. Not `Book`. Nothing at all -- and the name matches the class exactly.

The parameter wins

```
public Book(String title, String author) {
  title = title;        // compiles. runs. does nothing.
  author = author;
}
```

Inside the constructor, `title` means the *parameter* -- it is nearer. The field is still `null` when the constructor returns.

→ Watch the fields stay null in Java Visualizer.

`this` is the object you are inside of

```
public Book(String title, String author) {
  this.title = title;    // this.title is the FIELD
  this.author = author;  // title is the PARAMETER
}
```

`this` is a reference to whichever `Book` is being built right now. `this.title` can only mean the field.

`this` works in every instance method

```
public void checkOut() {
  this.checkedOut = true;
  this.daysLeft = 14;
}
```

No parameter to collide with here, so `this.` is optional. Write it anyway -- it says *field*, out loud.

Importantly: there is no `this` inside a `static` method. There is no object to point at.

Eclipse Exercise

Write the fields and the two-argument constructor.

TODOs 1 & 2 in `Book.java`.

Every field gets a value -- including the two that were never explicitly given.

Tophat Q1

A check on fields, constructors, and `this`

→ Open Tophat to vote.

One class, two ways in

Most books go straight to the shelf. Some arrive already on loan. Do we need seperate classes? **No!**

```
Book a = new Book("Dune");
Book b = new Book("Emma", "Jane Austen");
Book c = new Book("Hamlet", "Shakespeare", true);
```

→ Step through all three in Java Visualizer.

Java picks by the parameter list

Next to `Book(String title)`	Legal?
`Book(String t, String a)`	yes, 2 params
`Book(String t, boolean b)`	yes, new types
`Book(boolean b, String t)`	yes, new order
`Book(String author)`	no, same list

Name plus parameter types **in order** is the **signature**.

`this(...)` calls the matching constructor

```
public Book(String title, String author, boolean checkedOut) {
  this(title, author);          // run the 2-arg one first
  this.checkedOut = checkedOut;
}
```

`this.title` is a field. `this(...)` is a constructor call. Same word, and it must be the **first** statement in the constructor.

What is wrong here?

```
public Book(String title, String author) {
  this.title = title;
  this.author = author;
}

public Book(String title) {
  this.title = title;
}
```

Both compile. One of them leaves a `Book` in a state where `author` could be `null`. Constructors are a great time to specify defaults (if applicable).

Overloading is not just for constructors

What we were doing with constructors is known as **overloading**. We can also do it with methods.

```
public void renew() {
  this.daysLeft = 14;
}

public void renew(int days) {
  this.daysLeft = days;
}
```

Eclipse Exercise

TODOs 3 & 4 complete the overloading in `Book.java`.

Then run `Main` -- all four TODOs done, it compiles at last.

Tophat Q2

A check on overloading.

→ Open Tophat to vote.

Questions?